INTRODUCTION

In the contemporary digital era, the rapid proliferation of internet-enabled devices and the transition of essential services to online platforms have fundamentally altered the landscape of social and professional interaction. However, this technological evolution has also given rise to a sophisticated breed of criminal activity. Cybercrimes—ranging from financial fraud and identity theft to social media harassment and data breaches—now pose a significant threat to individuals and organizations alike. Despite the growing frequency of these incidents, many victims find the process of seeking legal recourse to be daunting, disorganized, and opaque. Traditional, paper-based reporting systems are often ill-equipped to handle the nuances of digital evidence, leading to delayed investigations and administrative bottlenecks.

The **Cybercrime Case Registration System** is a strategic technological intervention designed to address these challenges. It serves as a comprehensive, end-to-end digital solution that streamlines the lifecycle of a cybercrime complaint, from initial filing to final resolution. By providing a structured, user-centric interface, the platform empowers complainants to report incidents with precision, ensuring that critical data points—such as incident dates, crime categories, and detailed descriptions—are captured systematically. A core innovation of this system is its robust evidence-handling module, which allows for the secure upload and management of digital artifacts, providing investigators with the necessary tools to analyze cases with high technical integrity.

Beyond simple data entry, the system is engineered to foster transparency and trust between the public and law enforcement. Through a dedicated tracking module, users can monitor the progress of their cases in real-time using unique reference identifiers, eliminating the anxiety associated with traditional "black-hole" reporting processes. For administrators, the platform functions as a powerful management dashboard, enabling them to categorize cases, update investigation statuses, and maintain a high-integrity database of all reported crimes.

Ultimately, the Cybercrime Case Registration System is more than just a reporting tool; it is a framework for digital justice. By integrating modern web technologies with secure database management, the project aims to minimize human error, reduce response times, and provide a reliable, efficient, and accessible path for victims to report and resolve cyber-related offenses in an increasingly complex digital world.

OBJECTIVE

The primary goal of the **Cybercrime Case Registration System** is to provide a highly secure, efficient, and user-friendly digital environment for the reporting and management of cyber-related offenses. The specific objectives are as follows:

- **To Digitalize Case Management:** Replace outdated, manual, and paper-based reporting methods with a streamlined digital workflow that ensures data consistency, longevity, and easy retrieval.
- **To Enhance Reporting Accuracy:** Provide structured forms and validation logic that guide users to provide essential details—such as specific crime categories, incident dates, and precise descriptions—thereby reducing ambiguity in the initial report.
- **To Secure Evidence Handling:** Implement a reliable module for the upload and storage of digital evidence (images, documents, and videos), allowing investigators to access crucial artifacts without the risk of physical loss or corruption.
- **To Provide Real-Time Transparency:** Empower complainants by providing unique reference identifiers and a dedicated tracking interface, allowing them to monitor the investigation status (Pending, In Progress, or Resolved) at their convenience.
- **To Optimize Administrative Efficiency:** Equip law enforcement and administrators with a centralized dashboard to view, categorize, update, and manage multiple cases simultaneously, significantly reducing the administrative overhead.
- **To Automate Communication:** Utilize automated email services to provide immediate acknowledgement of receipts and periodic updates, ensuring the user is informed throughout the investigation lifecycle.
- **To Ensure Data Integrity:** Utilize a robust MySQL backend to store case records securely, preventing unauthorized access and ensuring that all legal documentation remains tamper-proof.

TOOLS & ENVIRONMENT

1. **Software:**

   Backend: Python 3.x (using the Flask Framework).

   Frontend: HTML5, CSS3, and JavaScript (ES6+).

   Database: MySQL (via XAMPP/Localhost)

   Code editor (VS Code)

2. **Hardware:**

   - 2 GB RAM
   - Mac processor/Window
   - Hard disk 64GB

ANALYSIS DOCUMENT

4.1 Software Requirements Specification (SRS)

The SRS establishes the "what" of the system before the "how."

1. Introduction:

Purpose: To provide a web-based, automated interface for reporting and tracking cybercrimes.

Scope: Covers user registration, case filing with multi-file evidence, admin management, and SMTP notifications.

2. Overall Description:

Product Perspective: A three-tier web application using Flask and MySQL.

User Characteristics: Complainants (General Public) and Administrators (Law Enforcement).

3. Specific Requirements:

Functional: * Generate unique Reference IDs.

Support up to 20 evidence file uploads per case.

Real-time status tracking and search filters for admins.

Non-Functional:

Security: Use of session-based authentication and input sanitization.

Performance: MySQL queries optimized for <2s response times.

Maintainability: Modular Python code for easy updates.

4.2 System Modeling (ER & Class Diagrams)

Entity-Relationship (ER) Diagram

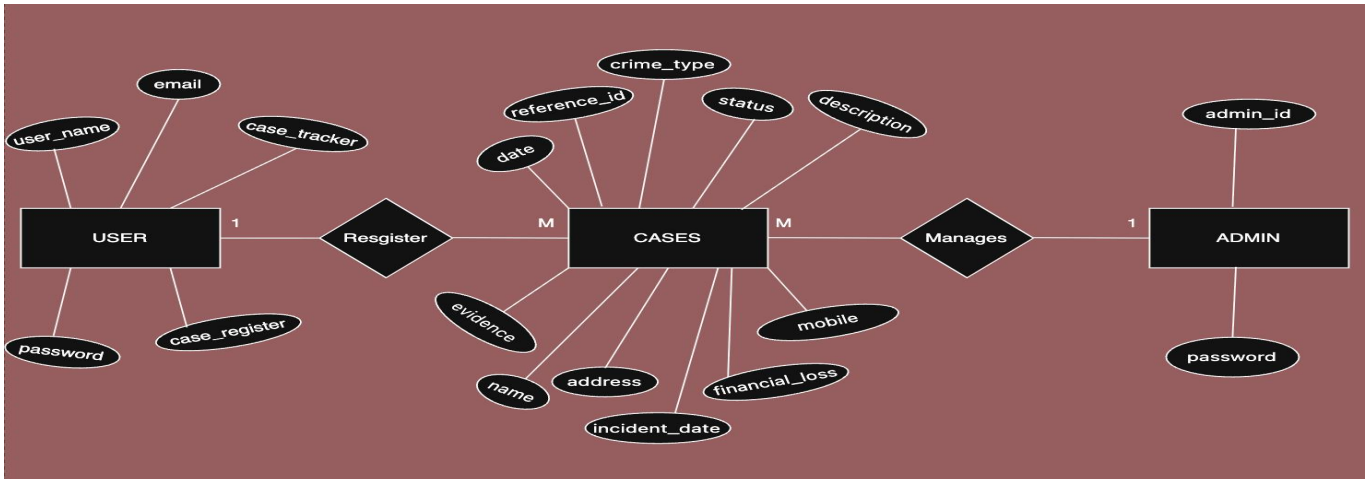

Figure: 4.1

Class Diagram

Figure: 4.2

4.3 Data Flow Diagrams (DFD)

DFD level 0

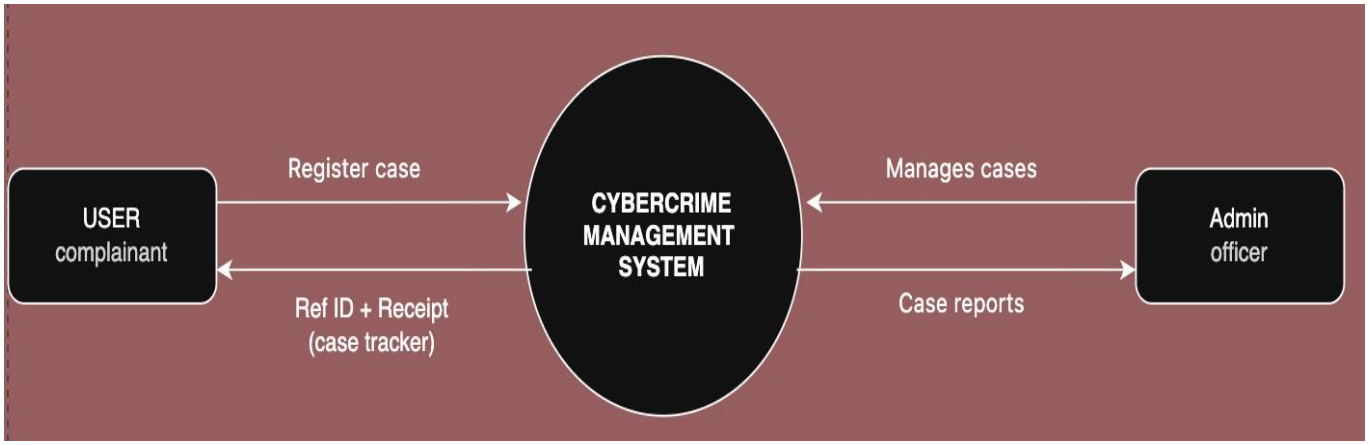

Figure: 4.3

DFD level 1

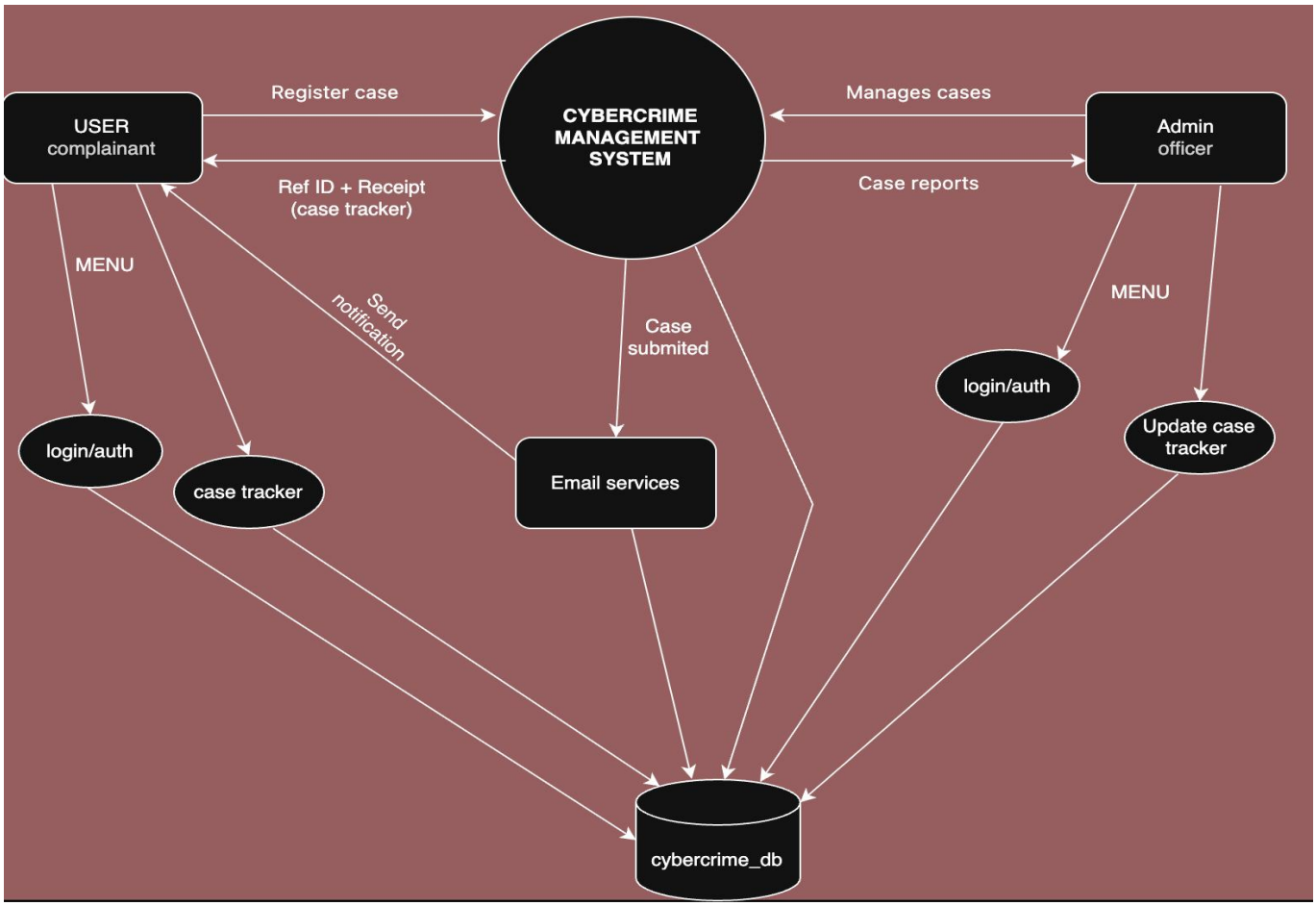

Figure: 4.4

4.4 Data Dictionary

The Data Dictionary provides a detailed breakdown of every field used in the MySQL database.

Field Name	Data Element	Data Type	Description
ref_id	Reference ID	VARCHAR(20)	Unique alphanumeric ID for case tracking.
comp_name	Complainant Name	VARCHAR(100)	Full name of the person filing the report.
crime_cat	Crime Category	ENUM/VARCHAR	Type of crime (e.g., Phishing, Identity Theft).
case_status	Status	VARCHAR(15)	Current stage: 'Pending', 'In Progress', 'Resolved'.
evidence_path	File Storage	LONGTEXT	Serialized paths of the 20 uploaded documents.
filed_at	Timestamp	DATETIME	The exact date and time the report was submitted.

Table 4.1(Data Dictionary)

DESIGN DOCUMENT

5.1 System Architecture & Modularization

The system is built using a **Three-Tier Architecture**, which separates the application into three logical and physical computing tiers:

1. **Presentation Tier (User Interface):** Developed with **HTML5**, **CSS3**, and **JavaScript**. It uses a modular component approach for the navigation bar, footer, and form elements to ensure consistency across the User and Admin panels.

2. **Application Tier (Process Logic):** Powered by **Flask (Python)**. The backend is modularized into specific functional routes:

   auth.py: Handles login, registration, and session security.

   reporting.py: Manages case submission logic and Reference ID generation.

   admin.py: Contains logic for status updates, searching, and record deletion.

3. **Data Tier (Storage):** A **MySQL** relational database handles persistent data storage.

5.2 Use Case Diagram

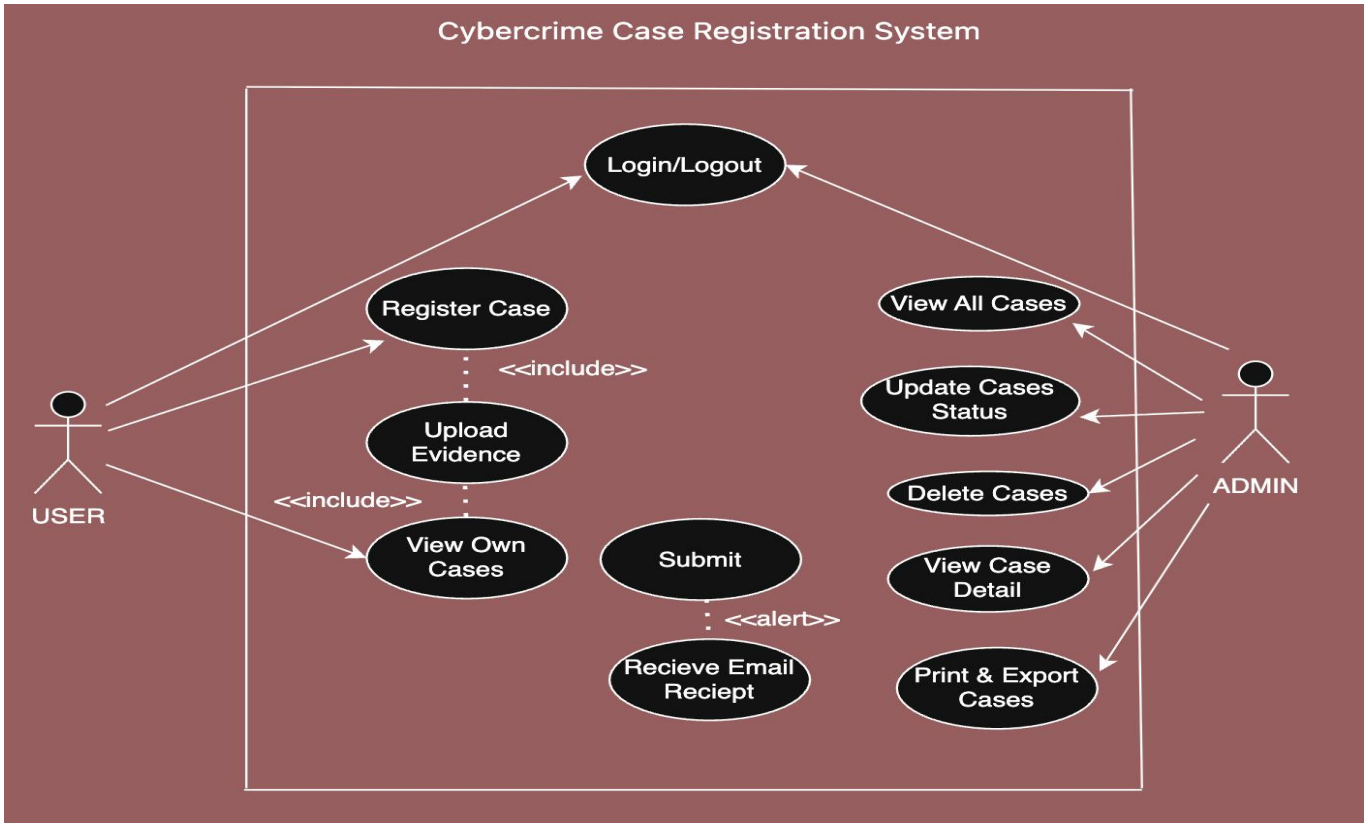

Figure: 5.1

5.3 Data Integrity & Constraints

Constraint Type	Implementation	Purpose
Entity Integrity	id is a Primary Key (Auto-increment).	Ensures every record is unique.
Referential Integrity	Foreign Keys link Evidence files to a specific reference_id.	Prevents "orphan" evidence files without a valid case.
Domain Integrity	crime_type uses a restricted ENUM list.	Ensures only valid crime categories are entered.
Uniqueness	reference_id and email (UNIQUE).	Prevents duplicate case filings
Data Validation	NOT NULL constraints on critical fields.	Ensures mandatory data (like incident date) is captured.

Table 5.1(Data Integrity & Constraints)

5.4 Procedural Design

The procedural design defines the step-by-step logic for core system operations.

Case Registration Procedure:

- **Input:** User fills the report form and attaches files.
- **Validation:** System checks name length (2+ parts), mobile (10 digits), and file count (≤ 20).
- **Processing:** * A unique **Reference ID** is generated using a timestamp-based hash.
- Files are saved to a secure /static/uploads/ directory.
- **Storage:** Case data and file paths are committed to the MySQL database.
- **Output:** An automated confirmation email is sent via SMTP, and a success receipt is displayed.

5.5 User Interface (UI) Design

The UI is designed for high-stress environments where clarity and speed are essential.

- **Typography:** Uses **Inter** for UI elements (buttons, inputs) and **Poppins** for headings to maintain a professional, modern aesthetic.
- **Color Palette:** A high-contrast palette (Deep Blue/White) is used to signify authority and trust, which is critical for a law enforcement application.
- **Responsive Layout:** A grid-based system ensures the dashboard is accessible on both desktop monitors (for Admins) and mobile devices (for Victims reporting on the go).
- **Interactive Elements:** * **Real-time Filters:** Admins can filter cases instantly without page reloads.
- **Progress Bars:** The Case Tracker uses a visual status bar (Pending → In Progress → Resolved) for immediate user understanding.

PROGRAM CODE

```
import os

import mysql.connector

import smtplib

from email.message import EmailMessage

from flask import Flask, render_template, request, redirect, url_for, session

from datetime import datetime, timedelta

app = Flask(__name__)

app.secret_key = "cyber_security_key"

app.config['MAX_CONTENT_LENGTH'] = 100 * 1024 * 1024

# --- CONFIGURATION FOR FILE UPLOADS ---

UPLOAD_FOLDER = 'static/uploads'

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

if not os.path.exists(UPLOAD_FOLDER):

os.makedirs(UPLOAD_FOLDER)

# --- MYSQL CONFIGURATION (XAMPP DEFAULT) ---

db_config = {

'host': 'localhost',

'user': 'root',

'password': '',

'database': 'cybercrime_db'

}

def get_db_connection():

"""Establishes connection to the MySQL database."""

return mysql.connector.connect(**db_config)
```

```
# 1. DATABASE INITIALIZATION

def init_db():

try:

# Step A: Connect to MySQL server (WITHOUT database name) to ensure DB exists

conn = mysql.connector.connect(

host=db_config['host'],

user=db_config['user'],

password=db_config['password']

)

cursor = conn.cursor()

cursor.execute(f"CREATE DATABASE IF NOT EXISTS {db_config['database']}")

conn.close()

# Step B: Create the table using the specific database

conn = get_db_connection()

cursor = conn.cursor()

cursor.execute('''

CREATE TABLE IF NOT EXISTS cases (

id INT AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(255) NOT NULL,

address TEXT,

email VARCHAR(255) NOT NULL,

mobile VARCHAR(20),

incident_date VARCHAR(50),

financial_loss VARCHAR(100),

crime_type VARCHAR(255) NOT NULL,
```

```
description TEXT,
evidence TEXT,
date VARCHAR(50),
reference_id VARCHAR(100) UNIQUE NOT NULL,
status VARCHAR(50) DEFAULT 'Pending'
)
''')
conn.commit()
conn.close()
print("    MySQL Database & Table Initialized.")
except Exception as e:
print(f"    Initialization Error: {e}")
# 2. HELPER FUNCTIONS
def send_email_notification(receiver_email, ref_id, name):
msg = EmailMessage()
msg.set_content(f"Dear {name},\n\nYour cybercrime case has been registered successfully.\nReference ID: {ref_id}\n\nPlease keep this ID for future inquiries.")
msg['Subject'] = 'Cybercrime Case Registration Successful'
msg['From'] = "gautambhardwaj05851@gmail.com"
msg['To'] = receiver_email
try:
server = smtplib.SMTP('smtp.gmail.com', 587)
server.starttls()
# Note: Ensure you use a Google "App Password" here, not your regular password
server.login("gautambhardwaj05851@gmail.com", "oawm kayg mvmk jxuh")
```

```
server.send_message(msg)

server.quit()

return True

except Exception as e:

print(f"     Email Error: {e}")

return False

# 3. ROUTES

@app.route("/")

def login_page():

return render_template("login.html")

@app.route("/login", methods=["POST"])

def login():

role = request.form.get("role")

password = request.form.get("password")

if role == "admin":

username = request.form.get("username", "").strip().lower()

if username == "admin" and password == "123":

session["role"] = "admin"

return redirect(url_for("admin_dashboard"))

return "Invalid Admin Credentials     "

elif role == "user":

if password == "user123":

session["role"] = "user"

session["user_name"] = request.form.get("username")

session["user_email"] = request.form.get("email")
```

```
action = request.form.get("user_action")

if action == "register":

return redirect(url_for("user_dashboard"))

return redirect(url_for("case_tracker"))

return "Invalid User Password     "

return redirect(url_for("login_page"))

@app.route("/tracker")

def case_tracker():

if session.get("role") != "user": return redirect(url_for("login_page"))

return render_template("tracker.html")

if ref_id:

cursor.execute("SELECT * FROM cases WHERE reference_id = %s", (ref_id.strip(),))

elif name:

cursor.execute("SELECT * FROM cases WHERE name LIKE %s", ('%' + name.strip() + '%',))

case = cursor.fetchone()

conn.close()

if not case:

return render_template("tracker.html", error="No case found     ")

return render_template("tracker.html", case=case)

@app.route("/admin")

def admin_dashboard():

if session.get("role") != "admin": return redirect(url_for("login_page"))

conn = get_db_connection()

cursor = conn.cursor(dictionary=True)

cursor.execute("SELECT * FROM cases ORDER BY id DESC")
```

```
cases = cursor.fetchall()

conn.close()

return render_template("admin.html", cases=cases)

@app.route("/view_case/<ref_id>")

def view_case(ref_id):

if session.get("role") != "admin": return redirect(url_for("login_page"))

conn = get_db_connection()

cursor = conn.cursor(dictionary=True)

cursor.execute("SELECT * FROM cases WHERE reference_id = %s", (ref_id,))

case = cursor.fetchone()

conn.close()

return render_template("view.html", case=case) if case else ("Not found", 404)

@app.route("/user")

def user_dashboard():

if session.get("role") != "user": return redirect(url_for("login_page"))

return render_template("user.html")

@app.route("/register", methods=["POST"])

def register_case():

if session.get("role") != "user": return redirect(url_for("login_page"))

incident_date_str = request.form.get("incident_date")

user_email = request.form.get("email")

user_name = request.form.get("name")

ref_id = datetime.now().strftime("%Y%m%d%H%M%S")

date_filed = datetime.now().strftime("%Y-%m-%d")

# Handling multiple file uploads
```

```
uploaded_files = request.files.getlist("evidence_image")

saved_filenames = []

for file in uploaded_files:

if file and file.filename != '':

unique_name = f"{ref_id}_{file.filename.replace(' ', '_')}"

file.save(os.path.join(app.config['UPLOAD_FOLDER'], unique_name))

saved_filenames.append(unique_name)

combined_evidence = f"URL: {request.form.get('evidence_url', 'No Link')} | Files: {',
'.join(saved_filenames)}"

conn = get_db_connection()

cursor = conn.cursor()

query = """INSERT INTO cases (name, address, email, mobile, incident_date,

financial_loss, crime_type, description, evidence, date, reference_id, status)

VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"""

data = (user_name, request.form.get("address"), user_email, request.form.get("mobile"),

incident_date_str, request.form.get("financial_loss"), request.form.get("crime_type"),

request.form.get("description"), combined_evidence, date_filed, ref_id, "Pending")

cursor.execute(query, data)

conn.commit()

conn.close()

send_email_notification(user_email, ref_id, user_name)

# Form data dictionary for the report card template

case_details = {

'name': user_name, 'address': request.form.get("address"), 'email': user_email,

'mobile': request.form.get("mobile"), 'incident_date': incident_date_str,
```

```
'financial_loss': request.form.get("financial_loss"), 'crime_type': request.form.get("crime_type"),
'description': request.form.get("description"), 'evidence': combined_evidence,
'date': date_filed, 'reference_id': ref_id, 'status': "Pending"
}
return render_template("report_card.html", case=case_details)
@app.route("/update_status/<ref_id>", methods=["POST"])
def update_status(ref_id):
if session.get("role") != "admin": return redirect(url_for("login_page"))
new_status = request.form.get("status")
conn = get_db_connection()
cursor = conn.cursor()
cursor.execute("UPDATE cases SET status = %s WHERE reference_id = %s", (new_status, ref_id))
conn.commit()
conn.close()
return redirect(url_for("admin_dashboard"))
@app.route("/delete_case/<ref_id>", methods=["POST"])
def delete_case(ref_id):
if session.get("role") != "admin": return redirect(url_for("login_page"))
conn = get_db_connection()
cursor = conn.cursor()
cursor.execute("DELETE FROM cases WHERE reference_id = %s", (ref_id,))
conn.commit()
conn.close()
return redirect(url_for("admin_dashboard"))
@app.route("/logout")
```

```
def logout():

session.clear()

return redirect(url_for("login_page"))

if __name__ == "__main__":

init_db()

# Using port 5001 to avoid default MacOS AirPlay port conflicts

app.run(debug=True, port=5001)

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Case Tracker</title>

<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600&display=swap"
rel="stylesheet">

<style>

body {

font-family: 'Poppins', sans-serif;

background-color: var(--bg-color);

margin: 0;

display: flex;

justify-content: center;

align-items: center;

min-height: 100vh;

color: var(--text-main);
```

```
}
h2 {
font-weight: 600;
margin-bottom: 25px;
color: var(--text-main);
font-size: 1.6rem;
}
input {
width: 100%;
padding: 12px 15px;
margin-bottom: 12px;
border-radius: 6px;
border: 1px solid #dcdde1;
background: #f8f9fa;
font-family: 'Poppins', sans-serif;
box-sizing: border-box;
outline: none;
font-size: 0.9rem;
}
button {
width: 100%;
padding: 12px;
background-color: var(--bg-color);
border: none;
border-radius: 6px;
```

```
color: white;
font-weight: 500;
font-family: 'Poppins', sans-serif;
font-size: 1rem;
cursor: pointer;
margin-bottom: 10px;
transition: background 0.2s;
}
.result strong {
color: #7f8c8d;
font-weight: 500;
}
/* PROGRESS STEPS */
.progress {
display: flex;
justify-content: space-between;
margin-top: 20px;
gap: 6px;
}
.step {
flex: 1;
text-align: center;
font-size: 0.75rem;
padding: 8px 0;
background: #ecececc;
```

```
border-radius: 4px;
color: #a0a0a0;
font-weight: 600;
text-transform: uppercase;
}
.step.active {
background-color: #2ecc71;
color: white;
}
.error {
color: #e74c3c;
background: #fdedec;
padding: 12px;
border-radius: 6px;
font-size: 0.85rem;
margin-bottom: 15px;
border: 1px solid #fadbd8;
}
.logout-link {
display: inline-block;
margin-top: 20px;
color: #95a5a6;
text-decoration: none;
font-size: 0.85rem;
}
```

```
.logout-link:hover {
color: var(--bg-color);
text-decoration: underline;
}
</style>
</head>
<body>
<div class="container">
<h2>Case Tracker</h2>
{% if error %}
<div class="error">{{ error }}</div>
{% endif %}
<form method="GET" action="/view_case_user">
<input type="text" name="ref_id" placeholder="Reference ID" required>
<button type="submit">Track by ID</button>
</form>
<form method="GET" action="/view_case_user">
<input type="text" name="name" placeholder="Search by Name" required>
<button type="submit">Track by Name</button>
</form>
{% if case %}
<div class="result">
<p><strong>Name</strong> <span>{{ case.name }}</span></p>
<p><strong>Crime</strong> <span>{{ case.crime_type }}</span></p>
<p><strong>Status</strong> <span>{{ case.status }}</span></p>
```

```
<div class="progress">
<div class="step {% if case.status == 'Pending' %}active{% endif %}">Pending</div>
<div class="step {% if case.status == 'In Progress' %}active{% endif %}">In Progress</div>
<div class="step {% if case.status == 'Closed' %}active{% endif %}">Closed</div>
</div>
</div>
{% endif %}
<a href="/logout" class="logout-link">Back to Dashboard</a>
</div>
</body>
</html>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Secure Login | Cybercrime Portal</title>
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&display=swap"
rel="stylesheet">
<style>
:root {
--primary: #1a73e8;
--accent: #64b5f6;
--glass: rgba(255, 255, 255, 0.9);
}
```

```
body {
font-family: 'Inter', sans-serif;
margin: 0;
height: 100vh;
display: flex;
align-items: center;
justify-content: center;
/* Professional Gradient Background */
background: radial-gradient(circle at top left, #1a73e8, #0d47a1);
overflow: hidden;
}
/* Abstract decorative circles for background depth */
.circle {
position: absolute;
border-radius: 50%;
background: rgba(255, 255, 255, 0.1);
z-index: 0;
}
.login-container {
position: relative;
z-index: 1;
background: var(--glass);
backdrop-filter: blur(10px);
padding: 50px 40px;
```

```
border-radius: 24px;
box-shadow: 0 20px 50px rgba(0,0,0,0.2);
width: 100%;
max-width: 400px;
border: 1px solid rgba(255,255,255,0.3);
}
.header {
text-align: center;
margin-bottom: 35px;
}
.header h1 {
font-size: 28px;
font-weight: 700;
color: #1d2939;
margin: 0;
letter-spacing: -1px;
}
.role-switch {
display: flex;
background: #f2f4f7;
padding: 4px;
border-radius: 12px;
margin-bottom: 30px;
}
.role-switch input { display: none; }
```

```
/* Highlight effect when radio is checked */
.role-switch input:checked + span {
background: white;
color: var(--primary);
box-shadow: 0 4px 12px rgba(0,0,0,0.05);
display: block;
padding: 10px;
border-radius: 8px;
}
.input-group {
margin-bottom: 20px;
}
.input-group label {
display: block;
font-size: 12px;
font-weight: 700;
color: #344054;
margin-bottom: 8px;
text-transform: uppercase;
}
input[type="text"], input[type="email"], input[type="password"] {
width: 100%;
padding: 14px;
border: 1.5px solid #d0d5dd;
border-radius: 12px;
```

```
font-size: 16px;
box-sizing: border-box;
transition: 0.2s;
background: white;
}
input:focus {
border-color: var(--primary);
outline: none;
box-shadow: 0 0 0 4px rgba(26,115,232,0.1);
}
.login-btn {
width: 100%;
padding: 16px;
background: var(--primary);
color: white;
border: none;
border-radius: 12px;
font-size: 16px;
font-weight: 600;
cursor: pointer;
transition: 0.3s;
margin-top: 10px;
}
.hidden { display: none; }
.footer-text {
```

```
text-align: center;
margin-top: 25px;
font-size: 12px;
color: #98a2b3;
}
</style>
</head>
<body>
<div class="circle" style="width: 300px; height: 300px; top: -100px; left: -100px;"></div>
<div class="circle" style="width: 200px; height: 200px; bottom: -50px; right: -50px;"></div>
<div class="login-container">
<div class="header">
<h1>PORTAL ACCESS</h1>
<p>Cybercrime Management System</p>
</div>
<form action="/login" method="POST">
<div class="role-switch">
<label>
<input type="radio" name="role" value="user" checked onclick="toggleUI()">
<span>USER</span>
</label>
<label>
<input type="radio" name="role" value="admin" onclick="toggleUI()">
<span>ADMIN</span>
</label>
```

```
</div>
<div class="input-group">
<label id="nameLabel">Full Name</label>
<input type="text" name="username" id="username" placeholder="Enter your name" required>
</div>
<div class="input-group" id="emailGroup">
<label>Email Address</label>
<input type="email" name="email" id="email" placeholder="name@example.com" required>
</div>
<div class="input-group hidden" id="passGroup">
<label>Password</label>
<input type="password" name="password" id="password" placeholder="••••••••">
</div>
<div id="actionGroup">
<label>Select Action:</label><br>
<input type="radio" name="user_action" value="register"> Case Registration<br>
<input type="radio" name="user_action" value="tracker"> Case Tracker<br>
</div>
<button type="submit" class="login-btn">Sign In</button>
</form>
<div class="footer-text">
Protected by 256-bit encryption &copy; 2026
</div>
</div>
<script>
```

```
function toggleUI() {
const role = document.querySelector('input[name="role"]:checked').value;
const emailGrp = document.getElementById('emailGroup');
const passGrp = document.getElementById('passGroup');
const nameLbl = document.getElementById('nameLabel');
const userInp = document.getElementById('username');
const actionGrp = document.getElementById('actionGroup'); //      NEW
passGrp.classList.remove('hidden');
document.getElementById('password').required = true;
if (role === 'admin') {
emailGrp.classList.add('hidden');
actionGrp.classList.add('hidden'); //      HIDE OPTIONS
document.getElementById('email').required = false;
nameLbl.innerText = "Admin ID";
userInp.placeholder = "Enter your Admin ID";
} else {
emailGrp.classList.remove('hidden');
actionGrp.classList.remove('hidden'); //      SHOW OPTIONS
document.getElementById('email').required = true;
nameLbl.innerText = "Full Name";
userInp.placeholder = "Enter your full name";
}
}
//      ADD THIS BLOCK (MAIN FIX)
```

```
if (role === 'user') {
const selectedAction = document.querySelector('input[name="user_action"]:checked');
if (!selectedAction) {
alert("Please select Case Registration or Case Tracker     ");
e.preventDefault();
return false;
}
console.log("Selected Action:", selectedAction.value); // debug
}
// --- EXISTING VALIDATION ---
if (role === 'user') {
const nameParts = name.split(/\s+/).filter(part => part.length > 0);
const isValidName = nameParts.length >= 2 && nameParts.every(part => part.length >= 2);
const lettersOnly = /^[A-Za-z\s]+$/.test(name);
if (!lettersOnly) {
alert("User Name should only contain letters and spaces.");
e.preventDefault();
return false;
}
if (!isValidName) {
alert("Users must enter a full name (e.g., 'John Doe'). Each part must be 2+ letters.");
e.preventDefault();
return false;
}
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
if (!emailPattern.test(email)) {
alert("Please enter a valid email address.");
e.preventDefault();
return false;
}
}
};
</script>
</body>
</html>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Case Receipt | Cybercrime Portal</title>
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet">
<style>
:root {
--primary: #1a73e8;
--success-bg: #f6fef9;
--success-border: #6ce9a6;
--text-main: #1d2939;
--text-sub: #667085;
```

```
--border: #eaecf0;
}

body {
font-family: 'Inter', sans-serif;
background-color: #fcfcfd;
margin: 0;
padding: 40px 20px;
display: flex;
justify-content: center;
}
.receipt-card {
background: white;
width: 100%;
max-width: 700px;
padding: 50px;
border-radius: 12px;
box-shadow: 0 12px 32px rgba(0,0,0,0.05);
border: 1px solid var(--border);
position: relative;
}
.receipt-card::before {
content: "OFFICIAL RECEIPT";
position: absolute;
top: 50%;
```

```
left: 50%;
transform: translate(-50%, -50%) rotate(-45deg);
font-size: 60px;
font-weight: 900;
color: rgba(0,0,0,0.02);
pointer-events: none;
white-space: nowrap;
}
.header {
text-align: center;
border-bottom: 2px solid var(--primary);
padding-bottom: 24px;
margin-bottom: 32px;
}
.ref-id {
font-family: 'Courier New', Courier, monospace;
font-size: 24px;
font-weight: 700;
color: #027a48;
margin: 8px 0;
}
.details-grid {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 24px;
```

```
}
.value {
font-size: 15px;
color: var(--text-main);
font-weight: 500;
}
.description-box {
grid-column: span 2;
background: #f9fafb;
padding: 20px;
border-radius: 8px;
margin-top: 10px;
font-size: 14px;
line-height: 1.6;
color: #344054;
}
.footer-note {
text-align: center;
font-size: 11px;
color: #98a2b3;
margin-top: 25px;
padding-top: 25px;
border-top: 1px solid var(--border);
line-height: 1.6;
width: 100%;
```

```
}
/* UPDATED SECTION START */
.action-btns {
display: flex;
flex-direction: row; /* Changed from column to row */
gap: 15px;
margin-top: 50px;
flex-wrap: wrap; /* Ensures buttons stack if screen is too small */
}
.btn {
flex: 1; /* Makes buttons equal width */
min-width: 160px;
padding: 16px;
border-radius: 10px;
font-weight: 700;
font-size: 15px;
cursor: pointer;
text-decoration: none;
transition: 0.3s cubic-bezier(0.4, 0, 0.2, 1);
border: none;
text-align: center;
}
/* UPDATED SECTION END */
.btn-secondary { background: #f2f4f7; color: #344054; }
.btn-primary { background: var(--primary); color: white; }
```

```
@media print {
body { background: white; padding: 0; }
.receipt-card { box-shadow: none; border: none; width: 100%; }
.action-btns { display: none; }
}
</style>
</head>
<body>
<div class="receipt-card">
<div class="header">
<h1>Cybercrime Reporting System</h1>
<p style="margin: 8px 0 0 0; color: var(--text-sub); font-size: 14px;">Official Acknowledgement of Complaint</p>
</div>
<div class="status-banner">
<div class="label" style="color: #027a48;">Complaint Registered Successfully</div>
<div class="ref-id">{{ case.reference_id }}</div>
<div style="font-size: 12px; color: #054f31;">Please quote this Reference ID for all future inquiries.</div>
</div>
<div class="details-grid">
<div class="data-item">
<div class="label">Complainant Name</div>
<div class="value">{{ case.name }}</div>
</div>
<div class="data-item">
```

```
<div class="label">Date of Filing</div>

<div class="value">{{ case.date }}</div>

</div>

<div class="data-item">

<div class="label">Email Address</div>

<div class="value">{{ case.email }}</div>

</div>

<div class="data-item">

<div class="label">Mobile Number</div>

<div class="value">{{ case.mobile }}</div>

</div>

<div class="data-item" style="grid-column: span 2;">

<div class="label">Category of Crime</div>

<div class="value" style="color: var(--primary);">{{ case.crime_type }}</div>

</div>

<div class="data-item" style="grid-column: span 2;">

<div class="label">Full Address</div>

<div class="value">{{ case.address }}</div>

</div>

<div class="description-box">

<div class="label" style="margin-bottom: 8px;">Incident Description</div>

{{ case.description }}

</div>

<div class="description-box" style="border-left: 4px solid var(--primary); background: #f0f7ff;">
```

```
<div class="label" style="margin-bottom: 12px; color: var(--primary);">Evidence / Supporting Information</div>

<div style="font-family: monospace; font-size: 13px; line-height: 1.8;">

{% if case.evidence %}

{% set evidence_parts = case.evidence.split('|') %}

{% for part in evidence_parts %}

<div style="padding: 4px 0; border-bottom: 1px solid rgba(26, 115, 232, 0.1);">

<strong>{{ part.split(':')[0] }}:</strong> {{ part.split(':')[1:] | join(':') }}

</div>

{% endfor %}

{% else %}

No evidence provided.

{% endif %}

</div>

</div>

</div>

<div class="action-btns">

<button onclick="window.print()" class="btn btn-primary">Print / Save as PDF</button>

<a href="/user" class="btn btn-secondary">Back to Dashboard</a>

</div>

<div class="footer-note">

This document serves as an official acknowledgement of your complaint.<br>

Please store this record safely. Our cybercrime investigation unit will review the details within 24-48 business hours.<br>

<strong>© 2026 Cybercrime Management System - Digital Integrity Division</strong>

</div>
```

```
</div>

</body>

</html>

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Dashboard | Cybercrime Registration</title>

<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet">

<style>

:root {

--primary: #1a73e8;

--bg: #f8fafd;

--card-bg: #ffffff;

--text-main: #202124;

--text-sub: #5f6368;

--border: #dadce0;

--secondary: #f1f3f4; /* For clear button */

}


body {

font-family: 'Inter', -apple-system, sans-serif;

background-color: var(--bg);
```

```
color: var(--text-main);
margin: 0;
padding: 40px 20px;
display: flex;
flex-direction: column;
align-items: center;
}
h1 {
font-size: 26px;
font-weight: 700;
color: var(--primary);
margin-top: 0;
margin-bottom: 30px;
text-align: center;
letter-spacing: -0.5px;
}
h2 {
font-size: 20px;
font-weight: 600;
color: var(--text-main);
margin-bottom: 20px;
}
.form-group {
margin-bottom: 20px;
}
```

```
/* Added for the side-by-side row */
.form-row {
display: flex;
gap: 20px;
margin-bottom: 20px;
}
.form-row .form-group {
flex: 1;
margin-bottom: 0;
}
label {
display: block;
font-size: 13px;
font-weight: 600;
text-transform: uppercase;
color: var(--text-sub);
margin-bottom: 8px;
letter-spacing: 0.5px;
}
input, select, textarea {
width: 100%;
padding: 14px;
border: 1.5px solid var(--border);
border-radius: 10px;
font-size: 15px;
```

```
font-family: inherit;
box-sizing: border-box;
transition: all 0.2s ease;
}
input[readonly] {
background-color: #f1f3f4;
color: #70757a;
border-style: solid;
cursor: not-allowed;
}button {
flex: 1;
padding: 16px;
border-radius: 10px;
font-weight: 600;
font-size: 16px;
cursor: pointer;
transition: all 0.3s ease;
border: none;
}
.btn-submit {
background-color: var(--primary);
color: white;
}
.btn-submit:hover {
background-color: #1557b0;
```

```
transform: translateY(-1px);
}
.btn-clear {
background-color: var(--secondary);
color: #3c4043;
}
.btn-clear:hover {
background-color: #e8eaed;
}
.history-card {
background: #ffffff;
border: 1px solid #eef2f6;
padding: 20px;
margin-bottom: 12px;
border-radius: 12px;
transition: box-shadow 0.2s;
}
.case-header {
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: 8px;
}
.crime-type {
font-weight: 700;
```

```
color: var(--primary);
}
.logout-link {
text-align: center;
display: block;
margin-top: 25px;
color: #d93025;
text-decoration: none;
font-weight: 600;
font-size: 15px;
}
hr {
border: 0;
border-top: 1px solid #f1f3f4;
margin: 30px 0;
}
</style>
</head>
<body>
<div class="container">
<h1>REGISTER CASE</h1>
<form action="/register" method="POST" id="caseForm" enctype="multipart/form-data">
<div class="form-group">
<label>Verified Full Name</label>
<input type="text" name="name" value="{{ session.user_name }}" readonly>
```

```
</div>
<div class="form-group">
<label>Registered Email</label>
<input type="email" name="email" value="{{ session.user_email }}" readonly>
</div>
<hr>
<div class="form-group">
<label>Residential Address*</label>
<input type="text" name="address" id="address" placeholder="Enter full address"
minlength="10" title="Please enter a full address (at least 10 characters)" required>
</div>
<div class="form-group">
<label>Mobile Number*</label>
<input type="tel" name="mobile" id="mobile" placeholder="10-digit mobile" pattern="[6789][0-9]{9}"
required>
</div>
<div class="form-row">
<div class="form-group">
<label>Date of Incident</label>
<input type="date" name="incident_date" id="incident_date" required>
</div>
<div class="form-group">
<label>Financial Loss (₹)</label>
<input type="number" name="financial_loss" id="financial_loss" placeholder="0" min="0">
</div>
```

```
</div>
<div class="form-group">
<label>Category of Incident</label>
<select name="crime_type" id="crime_type" required>
<option value="" disabled selected>Choose a category</option>
<option>Online Fraud</option>
<option>Phishing</option>
<option>Ransomware</option>
<option>Identity Theft</option>
<option>Hacking</option>
<option>Cyberstalking</option>
<option>Data Breach</option>
<option>Child Exploitation</option>
<option>Social Media Crime</option>
<option>Financial Fraud</option>
<option>Other</option>
</select>
</div>
<div class="form-group">
<label>Incident Details</label>
<textarea name="description" id="description" placeholder="Explain what happened..." rows="4"
minlength="10" required></textarea>
</div>
<form action="/register" method="POST" id="caseForm" enctype="multipart/form-data">
<div class="form-group">
```

```
<label>Evidence / Supporting Information</label>

<div style="margin-bottom: 15px;">

<label style="font-size: 11px; color: var(--primary);">Upload Supporting Evidences (Up to 20)</label>

<input type="file" id="filePicker" accept="image/*,video/*,audio/*,.pdf,.doc,.docx" style="padding: 10px;" multiple>

<div id="fileQueueDisplay" style="margin-top: 10px; display: flex; flex-wrap: wrap; gap: 8px;"></div>

<input type="file" name="evidence_image" id="evidence_image" multiple style="display: none;">

</div>

<div style="margin-bottom: 15px;">

<label style="font-size: 11px; color: var(--primary);">Relevant Links (URL)</label>

<input type="url" name="evidence_url" id="evidence_url" placeholder="https://example.com/evidence-link">

</div>

<label style="font-size: 11px; color: var(--primary);">Evidence Description</label>

<textarea name="evidence_desc" id="evidence" placeholder="Transaction IDs, account numbers, etc." rows="3"></textarea>

</div>

<div class="btn-row">

<button type="button" class="btn-clear" onclick="clearSpecificFields()">Clear Page</button>

<button type="submit" class="btn-submit">Submit Official Report</button>

</div>

</form>

</div>

<div class="container">

<h2>Your Filing History</h2>

{% if cases %}
```

```
{% for c in cases %}
<div class="history-card">
<div class="case-header">
<span class="crime-type">{{ c.crime_type }}</span>
<span class="case-date">{{ c.date }}</span>
</div>
<p style="margin: 0; color: #444; font-size: 14px; line-height: 1.5;">{{ c.description }}</p>
<div style="margin-top: 10px; font-size: 12px; color: #999;">Reference: #{{ c.reference_id }}</div>
</div>
{% endfor %}
{% else %}
<p style="text-align: center; color: #999; font-style: italic;">No cases registered under this email yet.</p>
{% endif %}
<a href="/logout" class="logout-link">Exit Secure Portal</a>
</div>
<script>
// --- FILE QUEUE LOGIC ---
let finalFileQueue = []; // This array keeps all your files safe
const filePicker = document.getElementById('filePicker');
const display = document.getElementById('fileQueueDisplay');
const actualInput = document.getElementById('evidence_image');
// Handle selecting new files
if (filePicker) {
filePicker.addEventListener('change', function() {
const selected = Array.from(filePicker.files);
```

```
let limitExceeded = false;
selected.forEach(file => {
// Check if adding this file stays within the 20-file limit
if (finalFileQueue.length < 20) {
// Prevent duplicates
if (!finalFileQueue.some(f => f.name === file.name)) {
finalFileQueue.push(file);
}
} else {
// If the limit of 20 is already reached
limitExceeded = true;
}
});
// Show alert if the user tried to add more than 20 files
if (limitExceeded) {
alert("You can only upload a maximum of 20 supporting documents.");
}
renderQueue();
filePicker.value = ""; // Reset picker so you can pick more files again
});
}
function renderQueue() {
display.innerHTML = "";
finalFileQueue.forEach((file, index) => {
const tag = document.createElement('div');
```

```
// Matches your existing Blue/Inter UI theme

tag.style = "background:#e8f0fe; color:#1a73e8; padding:5px 12px; border-radius:20px; font-size:12px; display:flex; align-items:center; gap:8px; border:1px solid #1a73e8; margin-bottom:5px;";

tag.innerHTML = `

<span>${file.name}</span>

<span style="cursor:pointer; color:#d93025; font-weight:bold; font-size:16px;" onclick="removeSpecificFile(${index})">×</span>

`;

display.appendChild(tag);

});

}

function removeSpecificFile(index) {

finalFileQueue.splice(index, 1); // Delete only the specific file clicked

renderQueue();

}

// --- DATE RESTRICTION LOGIC ---

window.onload = function() {

const dateInput = document.getElementById('incident_date');

const today = new Date();

const fifteenDaysAgo = new Date();

fifteenDaysAgo.setDate(today.getDate() - 15);

const formatDate = (date) => {

let d = new Date(date),

month = '' + (d.getMonth() + 1),

day = '' + d.getDate(),

year = d.getFullYear();
```

```
if (month.length < 2) month = '0' + month;
if (day.length < 2) day = '0' + day;
return [year, month, day].join('-');
};
if (dateInput) {
dateInput.max = formatDate(today);
dateInput.min = formatDate(fifteenDaysAgo);
}
};
// --- FORM SUBMISSION & VALIDATION ---
document.getElementById('caseForm').onsubmit = function(e) {
const address = document.getElementById('address').value;
const mobile = document.getElementById('mobile').value;
const description = document.getElementById('description').value;
// 1. Move files from our Queue to the actual hidden form input
const dataTransfer = new DataTransfer();
finalFileQueue.forEach(file => dataTransfer.items.add(file));
actualInput.files = dataTransfer.files;
// 2. Multi-File Validation
if (finalFileQueue.length === 0) {
alert("Please upload at least one evidence file.");
e.preventDefault();
return false;
}
if (finalFileQueue.length > 20) {
```

```
alert("You can only upload a maximum of 20 supporting documents.");
e.preventDefault();
return false;
}
// 3. Address Validation
if (address.trim().length < 10) {
alert("Please provide a more detailed address (minimum 10 characters).");
e.preventDefault();
return false;
}
// 4. Mobile Validation (Starting with 6,7,8,9)
const mobilePattern = /^[6789][0-9]{9}$/;
if (!mobilePattern.test(mobile)) {
alert("Please enter a valid 10-digit mobile number starting with 6, 7, 8, or 9.");
e.preventDefault();
return false;
}
// 5. Description Validation
if (description.trim().length < 10) {
alert("Please explain the incident in more detail (minimum 10 characters).");
e.preventDefault();
return false;
}
};
function clearSpecificFields() {
```

```
document.getElementById('caseForm').reset();

finalFileQueue = []; // Clear the file queue array

renderQueue(); // Update the UI

}

</script>

</body>

</html>

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Case Detail | #{{ case.reference_id }}</title>

<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet">

<style>

:root {

--primary: #1a73e8;

--bg: #f8fafd;

--text-main: #202124;

--text-sub: #5f6368;

--border: #dadce0;

--success: #1e8e3e;

}
```

```
body {
font-family: 'Inter', sans-serif;
background-color: var(--bg);
padding: 40px 20px;
color: var(--text-main);
margin: 0;
}
.report-card {
max-width: 850px;
margin: 0 auto;
background: white;
padding: 50px;
border-radius: 16px;
border: 1px solid var(--border);
box-shadow: 0 10px 30px rgba(0,0,0,0.05);
}
.header {
border-bottom: 2px solid var(--primary);
padding-bottom: 25px;
margin-bottom: 40px;
display: flex;
justify-content: space-between;
align-items: center;
}
.header h1 {
```

```
margin: 0;
font-size: 24px;
color: var(--primary);
text-transform: uppercase;
}
.status-badge {
background: #e6f4ea;
color: var(--success);
padding: 6px 12px;
border-radius: 20px;
font-size: 12px;
font-weight: 700;
display: inline-block;
margin-top: 8px;
}
.ref-badge {
font-weight: 700;
background: #f1f3f4;
padding: 12px 20px;
border-radius: 8px;
font-size: 18px;
border: 1px solid var(--border);
}
.section-title {
font-size: 13px;
```

```
font-weight: 700;
color: var(--primary);
text-transform: uppercase;
letter-spacing: 1px;
margin-bottom: 20px;
display: flex;
align-items: center;
}
.grid {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 25px;
}
.info-group label {
display: block;
font-size: 11px;
color: var(--text-sub);
margin-bottom: 6px;
font-weight: 700;
text-transform: uppercase;
}
.description-box {
line-height: 1.6;
background: #fcfcfc;
padding: 20px;
```

```
border-radius: 10px;

border: 1px solid #f1f3f4;

white-space: pre-wrap;

font-size: 14px;

color: #444;

}

* EVIDENCE GALLERY LAYOUT */

.evidence-container {

display: flex;

gap: 20px;

align-items: flex-start;

}

.thumbnail-sidebar {

width: 160px;

display: flex;

flex-direction: column;

gap: 12px;

max-height: 550px;

overflow-y: auto;

padding-right: 8px;

}

/* Custom Scrollbar for Sidebar */

.thumbnail-sidebar::-webkit-scrollbar-thumb {

background: var(--border);

border-radius: 10px;
```

```
}
.main-preview-area {
flex: 1;
background: #f1f3f4;
border-radius: 12px;
border: 1px solid var(--border);
min-height: 500px;
display: flex;
align-items: center;
justify-content: center;
overflow: hidden;
padding: 10px;
}
* EVIDENCE STYLING */
.evidence-grid {
display: flex;
flex-direction: column;
gap: 15px;
}
.evidence-item {
background: #fff;
border: 1px solid #eee;
padding: 15px;
border-radius: 8px;
}
```

```
.evidence-link {
color: var(--primary);
text-decoration: none;
font-weight: 600;
}
.evidence-img {
max-width: 100%;
height: auto;
border-radius: 8px;
margin-top: 10px;
border: 1px solid #ddd;
box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}
.action-btns {
margin-top: 50px;
display: flex;
gap: 15px;
justify-content: center;
}
.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: #1557b0; }
.btn-back { background: #fff; color: var(--text-main); border: 1px solid var(--border); }
@media print {
.action-btns { display: none; }
body { padding: 0; background: white; }
```

```
.report-card { box-shadow: none; border: none; width: 100%; padding: 0; }
}
</style>
</head>
<body>
<div class="report-card">
<div class="header">
<div>
<h1>Incident Report</h1>
<div class="status-badge">● RECORD VERIFIED</div>
</div>
<div class="ref-badge">#{{ case.reference_id }}</div>
</div>
<div class="section">
<div class="section-title">1. Complainant Profile</div>
<div class="grid">
<div class="info-group">
<label>Full Name</label>
<div class="value">{{ case.name }}</div>
</div>
<div class="info-group">
<label>Mobile Number</label>
<div class="value">{{ case.mobile }}</div>
</div>
<div class="info-group">
```

```
<label>Email Address</label>

<div class="value">{{ case.email }}</div>

</div>

<div class="info-group">

<label>Date Filed</label>

<div class="value">{{ case.date }}</div>

</div>

<div class="info-group full-width">

<label>Permanent Address</label>

<div class="value">{{ case.address }}</div>

</div>

</div>

</div>

<div class="section">

<div class="section-title">2. Crime Details</div>

<div class="grid">

<div class="info-group">

<label>Crime Category</label>

<div class="value" style="color: #d93025; font-weight: 700;">{{ case.crime_type }}</div>

</div>

<div class="info-group">

<label>Date of Incident</label>

<div class="value">{{ case.incident_date }}</div>

</div>

<div class="info-group">
```

```
<label>Reported Financial Loss</label>

<div class="value">₹{{ case.financial_loss if case.financial_loss else "0" }}</div>

</div>

<div class="info-group full-width">

<label>Detailed Description</label>

<div class="description-box">{{ case.description }}</div>

</div>

</div>

</div>

<div class="section">

<div class="section-title">3. Digital Evidence & Media</div>

<div class="evidence-box">

{% if case.evidence and "Files:" in case.evidence %}

{# 1. Display the URL and Notes part safely #}

<div style="margin-bottom: 20px; padding: 15px; background: #f8fafd; border: 1px solid var(--border);
border-radius: 8px;">

{% set main_parts = case.evidence.split(' | Files: ') %}

<div style="font-size: 14px; color: var(--text-main);">

<strong>Reference Info:</strong><br>

<span style="color: var(--text-sub);">{{ main_parts[0] }}</span>

</div>

</div>

{# 2. Extract and display the files #}

{% if main_parts|length > 1 %}

{% set file_list = main_parts[1].split(', ') %}
```

```
<div class="evidence-container">

<div class="thumbnail-sidebar" id="thumbnailList">

{% for filename in file_list %}

{% if filename and filename != "No Files" %}

<div class="thumbnail-item"

onclick="updatePreview('{{ url_for('static', filename='uploads/' + filename) }}', '{{ filename }}', this)"

style="cursor: pointer; border: 2px solid #ddd; border-radius: 10px; overflow: hidden; background: #fff; flex-shrink: 0; margin-bottom:10px;">

{% if filename.lower().endswith(('.jpg', '.jpeg', '.png', '.gif')) %}

<img src="{{ url_for('static', filename='uploads/' + filename) }}" style="width: 100%; height: 85px; object-fit: cover; display: block;">

{% elif filename.lower().endswith(('.mp4', '.mov', '.avi')) %}

<div style="height: 85px; display: flex; align-items: center; justify-content: center; background: #000; color: #fff; font-size: 10px;">►► VIDEO</div>

{% else %}

<div style="height: 85px; display: flex; align-items: center; justify-content: center; background: #f1f3f4; color: #5f6368; font-size: 10px; text-align: center; padding: 5px;">►► DOC</div>

{% endif %}

</div>

{% endif %}

{% endfor %}

</div>

<div class="main-preview-area" id="mainPreviewContainer">

<div id="previewContent" style="width: 100%; height: 100%; display: flex; align-items: center; justify-content: center;">

<p style="color: var(--text-sub);">Select a file to preview</p>
```

```
</div>
</div>
</div>
{% endif %}
{% else %}
<p style="color: #999; font-style: italic; text-align: center; padding: 20px;">No digital evidence or files attached to this report.</p>
{% endif %}
</div>
</div>
<div class="action-btns">
<a href="/admin" class="btn btn-back">Back to Dashboard</a>
<button onclick="window.print()" class="btn btn-primary">Print / Save PDF</button>
</div>
</div>
<script>
/**
* Updates the main preview area based on the file type selected from the left sidebar.
* @param {string} fileUrl - The path to the file in static/uploads
* @param {string} fileName - The original name of the file
* @param {HTMLElement} element - The thumbnail div that was clicked
*/
function updatePreview(fileUrl, fileName, element) {
const previewContent = document.getElementById('previewContent');
const ext = fileName.split('.').pop().toLowerCase();
```

```
// 1. UI Update: Manage the "Active" selection state in the sidebar
document.querySelectorAll('.thumbnail-item').forEach(item => {
item.style.borderColor = "#ddd"; // Default gray border
item.style.borderWidth = "2px";
item.style.boxShadow = "none";
item.style.transform = "scale(1)";
});
// Highlight the selected thumbnail with your primary blue
if (element) {
element.style.borderColor = "#1a73e8";
element.style.borderWidth = "3px";
element.style.boxShadow = "0 4px 12px rgba(26, 115, 232, 0.2)";
element.style.transform = "scale(1.02)"; // Slight lift effect
}
// 2. Content Update: Change the preview area content
if (['jpg', 'jpeg', 'png', 'gif', 'webp'].includes(ext)) {
// Image Preview
previewContent.innerHTML = `
<img src="${fileUrl}"
alt="${fileName}"
style="max-width: 95%; max-height: 550px; border-radius: 8px; box-shadow: 0 4px 20px
rgba(0,0,0,0.15); transition: opacity 0.3s ease-in;">`;
}
else if (['mp4', 'mov', 'avi', 'webm'].includes(ext)) {
// Video Preview
```

```
previewContent.innerHTML = `

<video controls style="max-width: 95%; max-height: 500px; border-radius: 12px; background: #000; box-shadow: 0 10px 30px rgba(0,0,0,0.3);">

<source src="${fileUrl}" type="video/${ext}">

Your browser does not support the video tag.

</video>`;

}

else {

// Document Preview (PDF, Doc, etc.)

previewContent.innerHTML = `

<div style="text-align: center; background: white; padding: 60px 40px; border-radius: 16px; border: 1px solid #dadce0; width: 80%; max-width: 400px; box-shadow: 0 4px 15px rgba(0,0,0,0.05);">

<div style="font-size: 80px; margin-bottom: 20px;">►►</div>

<p style="font-weight: 700; color: #202124; margin-bottom: 8px; font-size: 18px; word-break: break-all;">${fileName}</p>

<p style="color: #5f6368; font-size: 14px; margin-bottom: 30px;">Supporting Document</p>

<a href="${fileUrl}" download class="btn btn-primary"

style="text-decoration: none; display: inline-block; padding: 14px 28px; border-radius: 8px; font-weight: 600;">

Download to View File

</a>

</div>`;

}

}

// Auto-select the first file when page loads

window.addEventListener('load', () => {
```

```
const firstThumbnail = document.querySelector('.thumbnail-item');
if (firstThumbnail) {
firstThumbnail.click();
}
});
</script>
</body>
</html>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Admin Dashboard | Case Management</title>
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet">
<style>
:root {
--primary: #1a73e8;
--bg: #f4f7f9;
--text-main: #1d2939;
--text-sub: #667085;
--border: #eaecf0;
--danger: #d93025;
}
```

```
body {
font-family: 'Inter', sans-serif;
background-color: var(--bg);
margin: 0;
padding: 40px;
color: var(--text-main);
}
.admin-wrapper {
max-width: 1200px;
margin: 0 auto;
background: white;
border-radius: 12px;
box-shadow: 0 4px 20px rgba(0,0,0,0.05);
border: 1px solid var(--border);
overflow: hidden;
}
.header-bar {
padding: 24px 32px;
background: white;
border-bottom: 1px solid var(--border);
display: flex;
justify-content: space-between;
align-items: center;
}
```

```
table {
width: 100%;
border-collapse: collapse;
table-layout: auto; /* Changed from fixed to handle content better */
}
th {
background: #f9fafb;
padding: 16px 20px;
text-align: left;
font-size: 11px;
font-weight: 700;
text-transform: uppercase;
color: var(--text-sub);
border-bottom: 1px solid var(--border);
letter-spacing: 0.5px;
}
tbody tr:hover {
background-color: #fcfdfe;
}
.ref-id {
font-family: monospace;
font-weight: 700;
color: var(--text-main);
cursor: pointer;
}
```

```
.crime-badge {
display: inline-block;
padding: 4px 10px;
border-radius: 6px;
font-size: 12px;
font-weight: 500;
background: #eef4ff;
color: var(--primary);
border: 1px solid #d1e0ff;
}
/* Status Select Styling */
.status-select {
padding: 6px 10px;
border-radius: 6px;
border: 1px solid var(--border);
font-family: 'Inter';
font-size: 12px;
cursor: pointer;
font-weight: 600;
width: 130px; /* Fixed width prevents shifting */
outline: none;
}
.logout-btn {
display: inline-block;
margin-top: 30px;
```

```
padding: 10px 24px;
color: var(--danger);
text-decoration: none;
font-weight: 600;
border: 1px solid var(--danger);
border-radius: 8px;
transition: all 0.2s;
}
.action-link {
color: var(--primary);
text-decoration: none;
font-weight: 600;
font-size: 13px;
}
.delete-btn {
background: none;
border: none;
color: var(--danger);
font-weight: 600;
cursor: pointer;
padding: 0;
font-size: 13px;
}
</style>
</head>
```

```
<body>

<div class="admin-wrapper">

<div class="header-bar">

<h1>CASE MANAGEMENT SYSTEM</h1>

<div style="font-weight: 600; color: var(--primary);">

Showing {{ cases|length }} Total Reports

</div>

</div>

<div class="header-bar">

<div>

<h2 style="margin: 0; font-size: 22px;">Case Records</h2>

<p style="margin: 5px 0 0; color: var(--text-sub); font-size: 14px;">Monitor and manage reported incidents</p>

</div>

<div style="position: relative; width: 350px;">

<span style="position: absolute; left: 15px; top: 50%; transform: translateY(-50%); color: var(--text-sub); font-size: 18px;">??</span>

<input type="text" id="caseSearch" onkeyup="filterCases()" placeholder="Search by Name or Case ID..."

style="width: 100%; padding: 12px 15px 12px 45px; border-radius: 8px; border: 1px solid var(--border); font-family: inherit; font-size: 14px; outline: none; transition: 0.2s; box-sizing: border-box;">

</div>

</div>

<table>

<thead>

<tr>
```

```
<th style="width: 160px;">Ref ID</th>
<th>Complainant</th>
<th>Crime Category</th>
<th style="width: 150px;">Status Tracker</th>
<th>Date Registered</th>
<th style="width: 140px;">Actions</th>
</tr>
</thead>
<tbody>
{% if cases %}
{% for c in cases %}
<tr>
<td class="ref-id" onclick="window.location.href='/view_case/{{ c.reference_id }}'">
#{{ c.reference_id }}
</td>
<div style="font-weight: 600;">{{ c.name }}</div>
<div style="font-size: 12px; color: var(--text-sub);">{{ c.email }}</div>
</td>
<td onclick="window.location.href='/view_case/{{ c.reference_id }}'">
<span class="crime-badge">{{ c.crime_type }}</span>
</td>
<td>
<form action="/update_status/{{ c.reference_id }}" method="POST" style="margin: 0;">
<select name="status" onchange="this.form.submit()" class="status-select"
style="background: {% if c.status == 'Resolved' %}#e6f4ea{% elif c.status == 'In Progress' %}#fef7e0{%
```

```
color: {% if c.status == 'Resolved' %}#1e8e3e{% elif c.status == 'In Progress' %}#b45309{%
else %}#5f6368{% endif %};">

<option value="Pending" {% if c.status == 'Pending' %}selected{% endif %}>     Pending</option>

<option value="In Progress" {% if c.status == 'In Progress' %}selected{% endif %}>     In
Progress</option>

<option value="Resolved" {% if c.status == 'Resolved' %}selected{% endif %}>     Resolved</option>

</select>

</form>

</td>

<button type="submit" class="delete-btn">??     Delete</button>

</form>

</div>

</td>

</tr>

{% endfor %}

{% else %}

<tr>

<td colspan="6" class="empty-state">No cases found in the database.</td>

</tr>

{% endif %}

</tbody>

</table>

</div>

<center>

<a href="/logout" class="logout-btn">Secure Admin Logout</a>
```

```
</center>
<script>
function filterCases() {
// 1. Get the search input value
const input = document.getElementById("caseSearch");
const filter = input.value.toUpperCase();
const table = document.querySelector("table");
const tr = table.getElementsByTagName("tr");
// 2. Loop through all table rows (starting from index 1 to skip header)
for (let i = 1; i < tr.length; i++) {
// Skip the "empty state" row if it exists
if (tr[i].classList.contains('empty-state')) continue;
// Get Reference ID (1st column) and Name (2nd column)
const refIdCell = tr[i].getElementsByTagName("td")[0];
const nameCell = tr[i].getElementsByTagName("td")[1];
if (refIdCell || nameCell) {
const refIdText = refIdCell.textContent || refIdCell.innerText;
const nameText = nameCell.textContent || nameCell.innerText;
// 3. Check if the input matches either the ID or the Name
if (refIdText.toUpperCase().indexOf(filter) > -1 ||
nameText.toUpperCase().indexOf(filter) > -1) {
tr[i].style.display = ""; // Show row
} else {
tr[i].style.display = "none"; // Hide row
}
```

```
}
}
}
// Add a little focus effect for the search bar
const searchInput = document.getElementById('caseSearch');
searchInput.addEventListener('focus', () => {
searchInput.style.borderColor = 'var(--primary)';
searchInput.style.boxShadow = '0 0 0 3px rgba(26, 115, 232, 0.1)';
});
searchInput.addEventListener('blur', () => {
searchInput.style.borderColor = 'var(--border)';
searchInput.style.boxShadow = 'none';
});
</script>
</body>
</html
```

TESTING & VALIDATIONS

The testing phase ensures that the **Cybercrime Case Registration System** is robust, secure, and performs as expected under various conditions. A multi-layered testing strategy was employed to verify both individual components and the integrated system.

7.1 Testing Methodologies

Testing Type	Description	Scope in Project
Unit Testing	Testing individual functions or modules in isolation.	Validating the Reference ID generation logic and individual SQL query strings in app.py.
White Box Testing	Testing the internal structures or workings of an application (Code-level).	Verifying loops, conditional statements, and database connection handling within the Flask backend.
Black Box Testing	Testing functional requirements without looking at the internal code.	Entering data into the user.html form to ensure the expected Case Receipt is generated.
Integration Testing	Testing the interaction between different modules.	Ensuring the Frontend (HTML) correctly communicates with the Backend (Python) and updates the MySQL database.
System Testing	Testing the complete and integrated software.	Executing a full end-to-end flow: Login → Case Registration → Evidence Upload → Admin Status Update → User Tracking.

Table 7.1(Methodologies)

7.2 Testing & Validation Table

This table details specific test cases, the input provided, and the observed validation behavior.

Test Case ID	Feature Tested	Input Provided	Expected Result	Status
TC-01	User Name Validation	"J" (Single Character)	System shows alert: "Name must be at least 2 characters."	**Pass**
TC-02	Email Validation	"user@gmail" (No TLD)	System prevents submission; requests valid email format.	**Pass**
TC-03	Mobile Validation	"5551234567" (Starts with 5)	System rejects; must start with 6, 7, 8, or 9.	**Pass**
TC-04	Evidence Limit	21 Files Uploaded	Alert: "Maximum 20 documents allowed."	**Pass**
TC-05	Reference Search	Valid ID (e.g., CR-2026-001)	Correct case details displayed in Tracker.	**Pass**
TC-06	Status Update	Change 'Pending' to 'Resolved'	Database updates and Tracker reflects 'Resolved'.	**Pass**

Table 7.2(Test & Validation)

INPUT & OUTPUT SCREENS

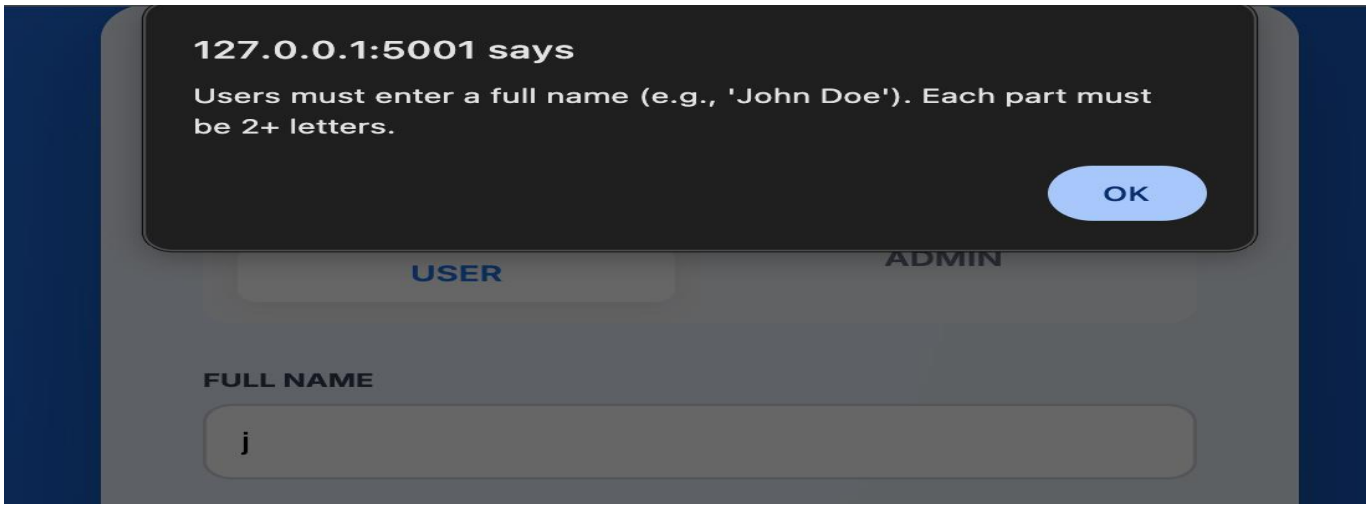

Figure8.1(User Name Validation)

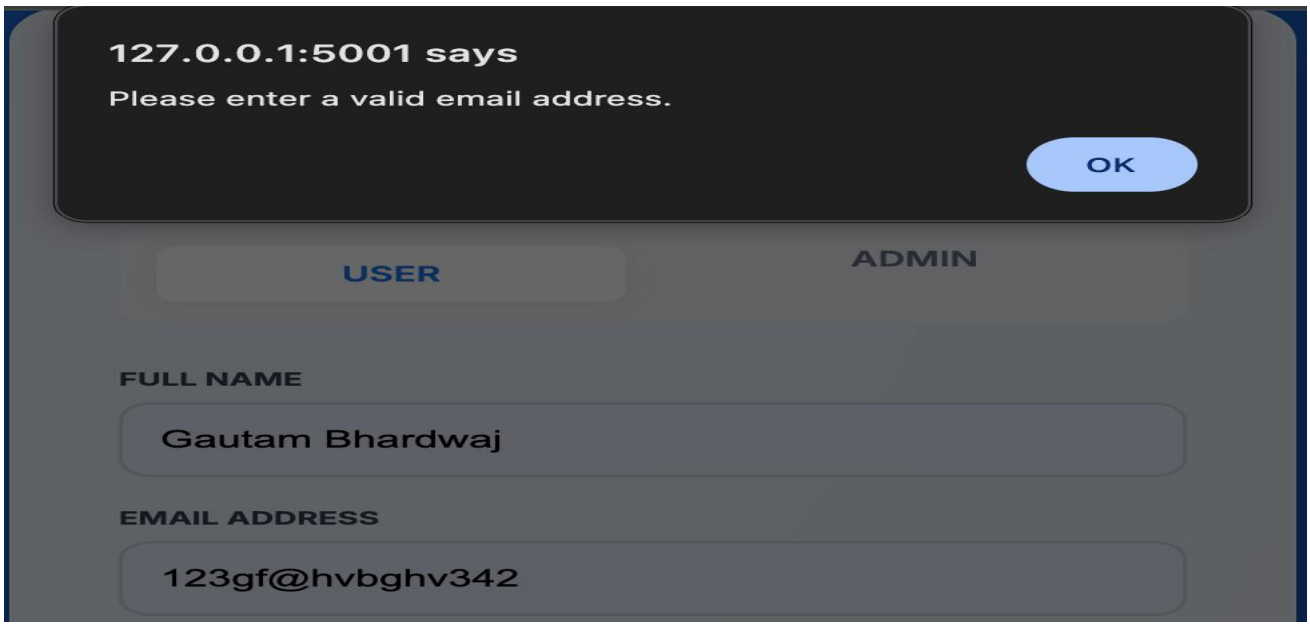

Figure8.2(Email Validation)

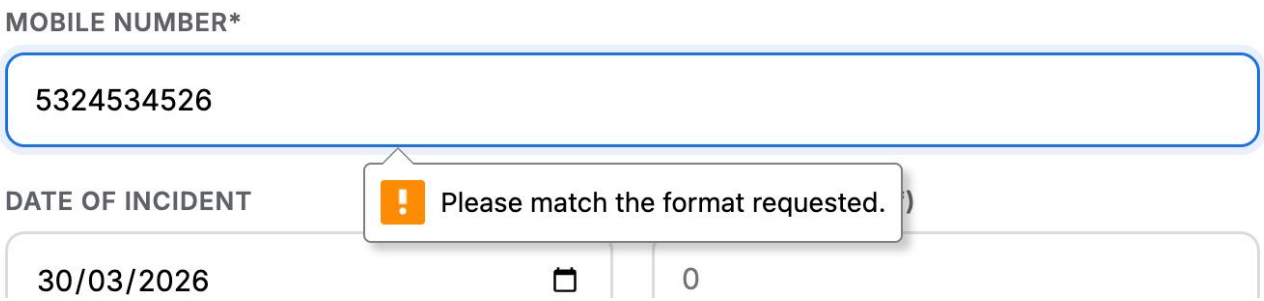

Figure8.3(Mobile Validation)

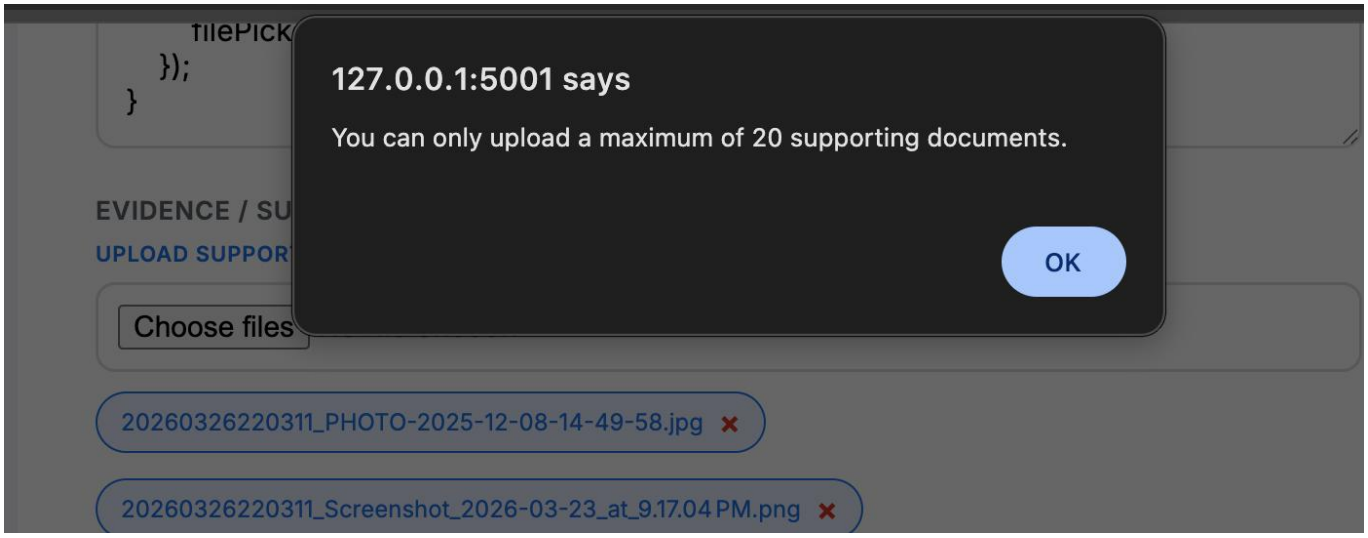

Figure8.4(Evidence Limit)

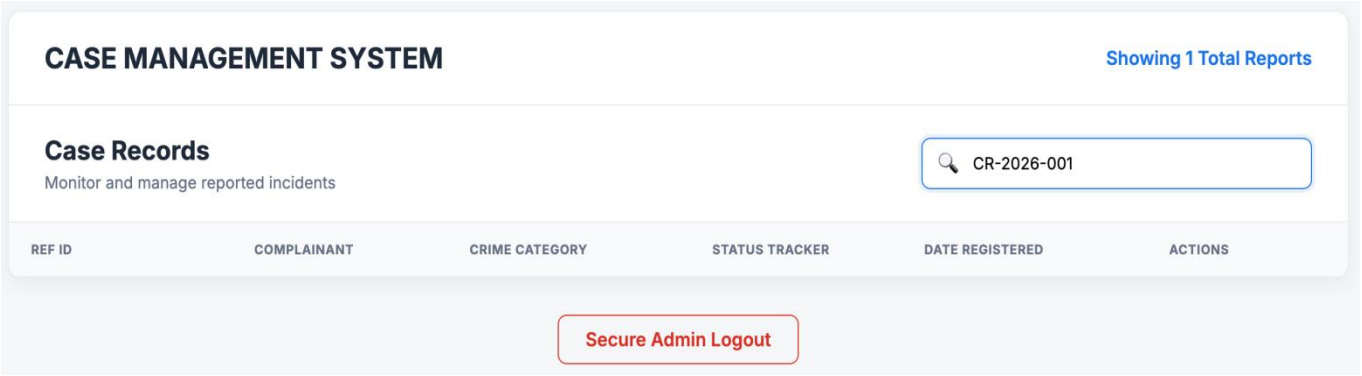

Figure8.5(Reference Search)

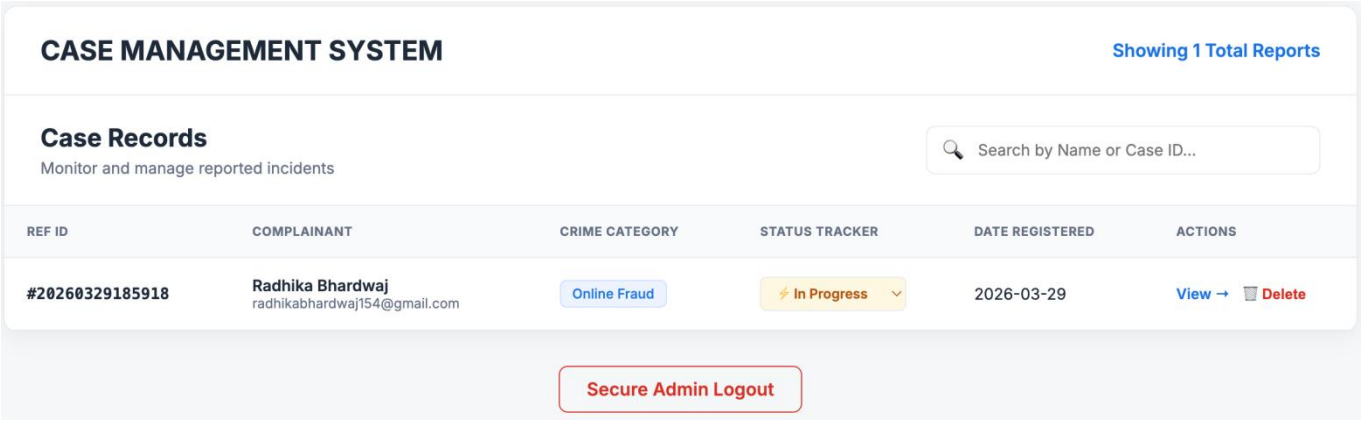

Figure8.6(Status Update)

LIMITATIONS OF THE PROJECT

While the **Cybercrime Case Registration System** significantly improves upon manual reporting methods, certain technical and operational limitations exist in the current version:

- **Dependency on Internet Connectivity:** As a web-based platform, the system requires a stable internet connection for case submission and real-time tracking. Users in remote areas with poor connectivity may face challenges in uploading large evidence files.
- **SMTP Service Limitations:** The current automated email notification system relies on Gmail SMTP. High volumes of reports may hit daily relay limits, and emails may occasionally be flagged as spam by the recipient's mail server.
- **Storage Constraints:** The system is currently configured to handle up to 20 files per case with a maximum total size of 100MB. Extremely large video files or high-resolution forensic images may require further compression.
- **Lack of Multilingual Support:** The interface is currently available only in English. This may limit accessibility for complainants who are more comfortable communicating in regional languages.
- **Fixed Database Schema:** The transition to MySQL has improved performance, but the schema is currently fixed. Adding new crime categories or custom fields requires manual database migration by a developer.
- **No Integrated Chat Support:** Communication between the admin and the user is currently one-way (via status updates and email). There is no real-time chat feature for investigators to ask follow-up questions directly within the portal.
- **Less Secure:** The system currently lacks advanced security features such as strong encryption, multi-factor authentication, and robust threat detection, making it vulnerable to potential cyber threats.

FUTURE APPLICATION OF THE PROJECT

The **Cybercrime Case Registration System** is designed with a scalable architecture that allows for significant enhancements as technology and cyber-legal requirements evolve. The following features are planned for future versions of the platform:

- **Artificial Intelligence for Evidence Analysis:** Integrating AI and Machine Learning models to automatically scan uploaded images and documents for common patterns in cybercrime, such as identifying phishing URLs or detecting manipulated/deepfake media.
- **Agentic AI for Automated Notifications:** Implementing AI agents to provide intelligent, personalized updates to complainants. This would include proactive reminders for missing documentation and automated status alerts via SMS or WhatsApp integration.
- **Bilingual and Multilingual Support:** To ensure inclusivity, the interface will be expanded to support multiple regional languages, allowing users from diverse linguistic backgrounds to report crimes with ease and clarity.
- **Geographical Crime Mapping:** Integrating a GIS (Geographic Information System) module to visualize crime hotspots. This would provide administrators with a heatmap of reported incidents, helping law enforcement allocate resources more effectively to high-risk areas.
- **Blockchain for Data Integrity:** Implementing a blockchain-based ledger to store case histories. This would create an immutable, tamper-proof record of every investigation step, ensuring the highest level of legal evidentiary standards.
- **Integrated Chatbot Assistance:** Adding a 24/7 AI-powered virtual assistant to help users categorize their crimes correctly and provide immediate guidance on "first-response" actions (e.g., how to freeze a bank account or secure a social media profile) after an incident.